Tugas Perancangan Perangkat Lunak F

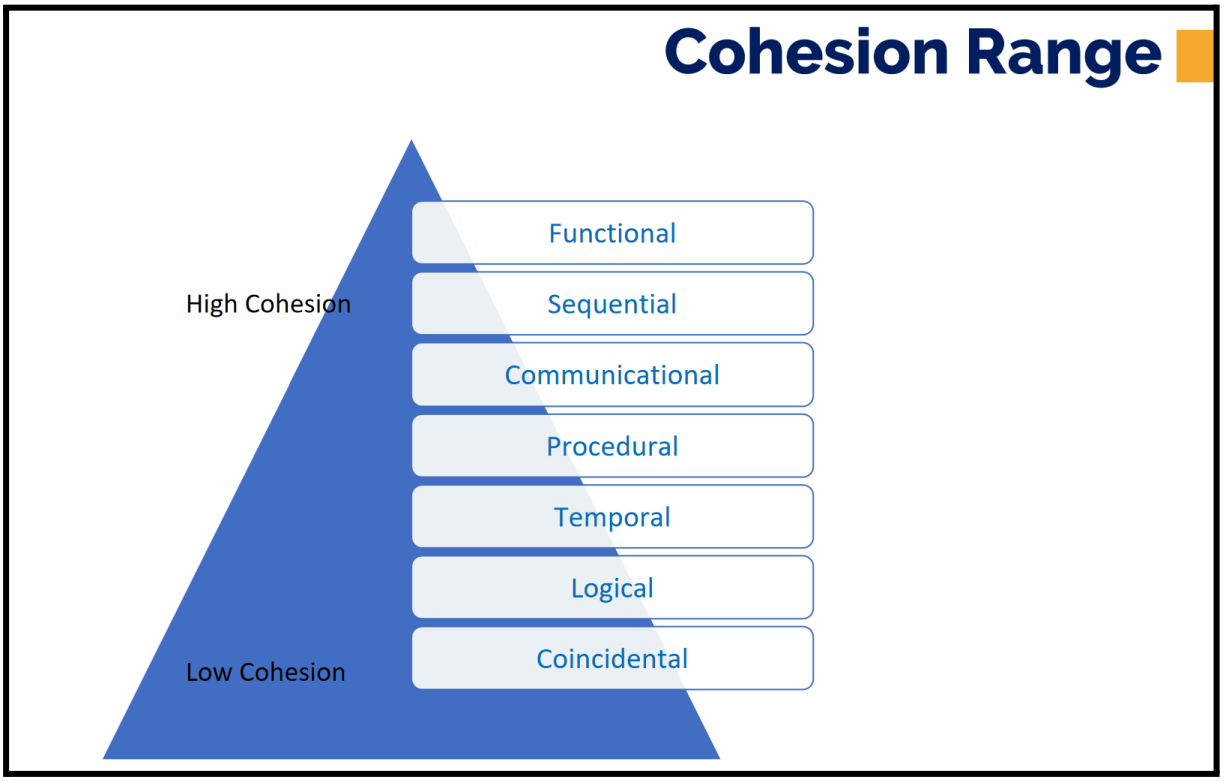
Kelompok "Pasukan Cabang PPL"

- Qurrata Ayun Kamil (5025241031)
- Muhammad Adi Anugerah Arrahman (5025241118)
- Mikail Ibrahim Hakim (5025241046)
- Maulana Ikhsan (5025241163)
- Adrian Aziz Santoso (5025221229)

A. Kohesi

Kohesi adalah ukuran kekuatan relasi internal elemen-elemen dalam satu modul, di mana tingkat kohesi yang tinggi (fungsional) menandakan modul tersebut fokus mengerjakan satu tugas saja.

- **Kegunaan:** Mengukur seberapa erat keterkaitan dan kerja sama fungsi-fungsi di dalam satu modul atau komponen untuk mencapai tujuan bersama.
- **Tujuan:** Membentuk kohesi tinggi di mana tiap satu modul hanya memiliki satu tanggung jawab atau tugas.
- **Contoh:** Sebuah modul khusus menangani autentikasi (dari *login, logout,* hingga penggantian *password*) memiliki kohesi fungsional yang tinggi.
- **Jenis (diurut dari terendah ke tertinggi):** *Coincidental, Logical, Temporal, Procedural, Communicational, Sequential, Functional*.



1. **Jenis-jenis Kohesi (diurut dari terendah ke tertinggi)**

- **Kohesi *Coincidental* (Terendah):** Komponen dikelompokkan bersama tanpa hubungan yang jelas, hanya kebetulan bersama-sama.
- **Kohesi Logis:** Elemen dikelompokkan karena melakukan aktivitas logis yang mirip, meskipun fungsinya berbeda (misal: modul *input* yang menangani *mouse* dan *keyboard* sekaligus).
- **Kohesi Temporal:** Elemen dikelompokkan karena dieksekusi pada waktu yang sama (misal: inisialisasi, *start-up*, atau *shutdown*).
- **Kohesi Prosedural:** Elemen dikelompokkan berdasarkan urutan eksekusi tertentu.
- **Kohesi Komunikasional:** Elemen beroperasi pada data *input* atau *output* yang sama.
- **Kohesi Sekuensial:** *Output* dari suatu elemen digunakan sebagai *input* untuk elemen berikutnya.
- **Kohesi Fungsional (Tertinggi):** Semua elemen berkontribusi langsung pada satu tugas secara terfokus, menjadikannya jenis kohesi terbaik.

2. **Contoh Kasus Kohesi: *E-Commerce***

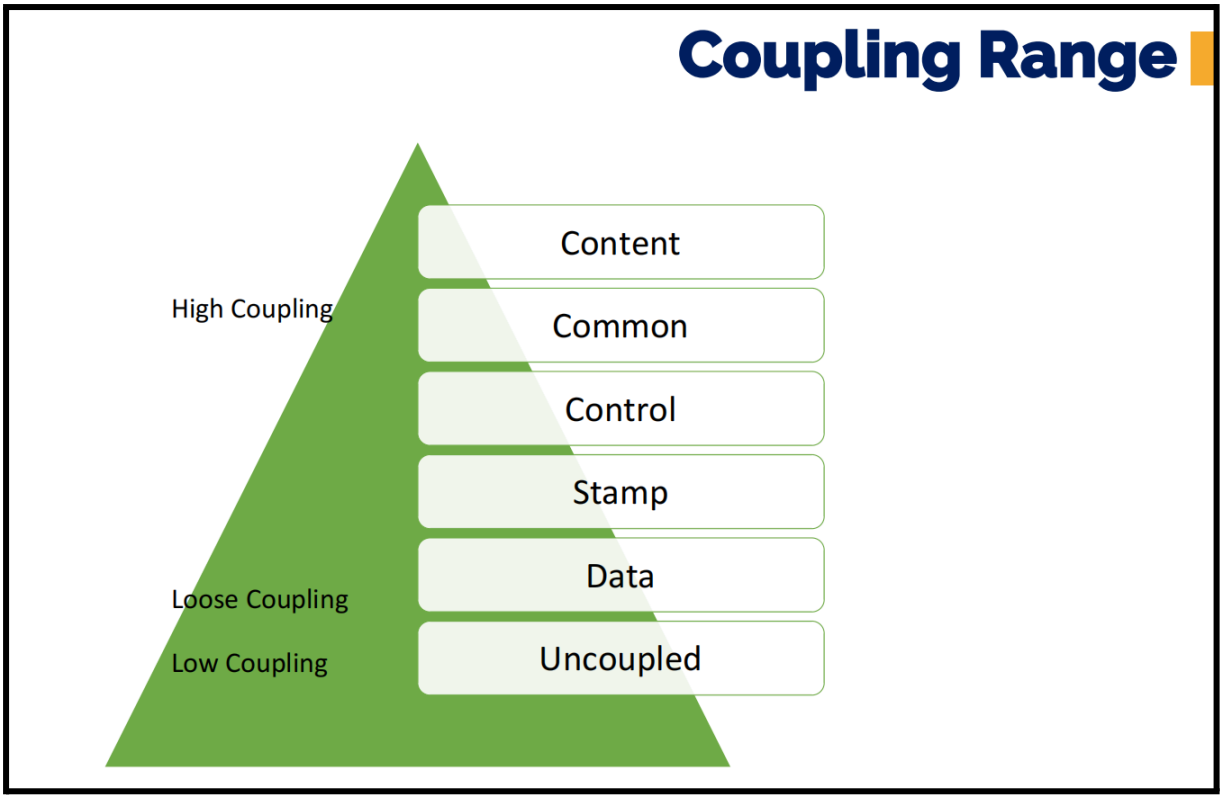
Jenis Kohesi	Contoh Penerapan dalam Modul *E-Commerce*	Penjelasan
Kohesi Fungsional (Tertinggi)	Modul Pembayaran (*Payment Gateway Integration*)	Modul ini hanya fokus pada satu tugas: memproses transaksi pembayaran (mengirim data menuju *payment gateway*, menerima respons sukses / gagal, dan memperbarui status pesanan). Semua fungsi di dalamnya terikat erat untuk satu tujuan tunggal ini.
Kohesi Sekuensial	Modul Pemrosesan Pesanan (*Order Processing*)	Modul yang terdiri dari serangkaian langkah yang berurutan, di mana *Output* dari langkah 1 (stok terverifikasi) menjadi *input* untuk langkah 2: 1. Verifikasi stok 2. Mengurangi stok 3. Membuat catatan pesanan.
Kohesi Komunikasional	Modul Manajemen Data Produk	Sebuah modul yang menangani semua operasi yang menggunakan data produk yang sama, seperti menampilkan detail produk di *front-end*, memperbarui stok di *back-end*, dan menghasilkan laporan inventaris. Semua fungsi beroperasi pada *data set* produk yang sama.

Kohesi Prosedural	Modul *Checkout Flow*	Sekumpulan fungsi yang dieksekusi dalam urutan langkah yang telah ditentukan untuk menyelesaikan proses *checkout*: 1. Memilih alamat 2. Memilih metode pengiriman 3. Menerapkan kode diskon 4. Menampilkan total akhir.
Kohesi Temporal	Modul Inisialisasi Sesi Pengguna (*Session Initialization*)	Fungsi-fungsi yang dijalankan bersamaan saat pengguna pertama kali mengakses situs atau *login*, seperti: memeriksa *cookie*, mengatur *session variable*, dan memuat preferensi tampilan pengguna.
Kohesi Logis	Modul Penanganan Input Pengguna (*User Input Handler*)	Modul yang menangani semua jenis input dari pengguna, seperti: Validasi input *form* pendaftaran, Penanganan *event* klik tombol, dan Pemrosesan input dari *chat support*. Meskipun fungsinya mirip (menangani input), tugas spesifiknya berbeda-beda.
Kohesi *Coincidental* (Terendah)	Modul Utilitas Campuran	Sebuah modul yang berisi fungsi yang tidak berhubungan sama sekali, misalnya fungsi untuk mengirim *email* notifikasi pesanan, fungsi untuk mengkompres gambar produk, dan fungsi untuk menghitung pajak. Ketiganya hanya ada bersama karena kebetulan ditempatkan dalam satu *file* atau *class* utilitas yang tidak terstruktur dengan baik.

B. *Coupling*

Dikenal sebagai "*inter-module binding*", *coupling* merujuk pada tingkat hubungan antara beberapa modul / paket / komponen yang saling bergantung satu sama lain.

- **Kegunaan / tujuan:** Mencapai *loose coupling* (ketergantungan rendah) untuk menciptakan sistem yang modular, mempermudah perawatan, meningkatkan fleksibilitas, dan memungkinkan pengembangan tim secara independen.
- **Manfaat:** *Loose coupling* (*coupling* yang rendah) memiliki manfaat berupa peningkatan kemandirian modul, kemudahan dalam pengujian/pemeliharaan, peningkatan *reusability*, stabilitas sistem, kemudahan pengembangan tim.
- **Jenis (diurut dari terendah ke tertinggi):** *Data, Stamp, Control, Common, Content*.



1. Jenis-jenis *Coupling* (diurut dari terendah ke tertinggi)

- ***Data Coupling* (Terendah):** Ketika modul-modul saling berbagi data primitif di antara mereka.
- ***Stamp Coupling*:** Ketika modul-modul berbagi data komposit atau struktural di antara mereka dan data tersebut harus berupa struktur data non-global. Semislal, meneruskan variabel objek atau struktur dalam komponen React.
- ***Control Coupling*:** Terjadi ketika data dari satu modul digunakan untuk mengarahkan struktur eksekusi instruksi di modul lain.
- ***External Coupling*:** Ketika dua modul berbagi tipe data eksternal di luar perangkat lunak, seperti protokol komunikasi, antarmuka perangkat.
- ***Common Coupling*:** Ketika dua modul berbagi data global yang sama dan bergantung padanya, seperti manajemen *state* dalam *framework* JavaScript.
- ***Content Coupling* (Tertinggi):** Terjadi ketika dua modul berbagi kode dan dapat memodifikasi data modul lain, ini adalah jenis *coupling* terburuk dan harus dihindari.

2. Contoh Kasus *Coupling*: *E-Commerce*

Jenis *Coupling*	Contoh Penerapan dalam Modul *E-Commerce*	Penjelasan
Data Coupling	Modul `Verifikasi_Stok` dan `Buat_Pesanan`	Modul `Buat_Pesanan` hanya memanggil fungsi di `Verifikasi_Stok` dan hanya mengirimkan `product_id` dan `quantity` (data primitif) sebagai parameter. Modul `Verifikasi_Stok` hanya mengembalikan `is_available` (boolean).
Stamp Coupling	Modul `Proses_Pembayaran` dan `Update_Alamat_Pengiriman`	Modul `Proses_Pembayaran` menerima seluruh objek `CustomerProfile` (yang berisi nama, alamat lengkap, nomor telepon, preferensi notifikasi, dll.) padahal modul tersebut hanya memerlukan `shipping_address` dan `billing_address` untuk melanjutkan transaksi.
Control Coupling	Modul `Proses_Pengiriman`	Modul `Buat_Pesanan` memanggil fungsi di `Proses_Pengiriman` dan mengirimkan *flag* `is_express: true` atau `is_express: false`. Modul `Proses_Pengiriman` kemudian menggunakan *flag* tersebut untuk mengarahkan alur eksekusinya (misalnya, memilih kurir A jika *true*, atau kurir B jika *false*).
External Coupling	Modul `Integrasi_Kurir` dan `Payment_Gateway`	Modul `Integrasi_Kurir` dan `Proses_Pembayaran` keduanya bergantung pada **protokol komunikasi** (misalnya, REST API dengan format JSON tertentu) dan **antarmuka** yang ditetapkan oleh penyedia layanan logistik eksternal atau penyedia *payment gateway*. Perubahan pada API eksternal akan memengaruhi kedua modul ini.
Common Coupling	Manajemen Global pada Status Keranjang Belanja	Beberapa modul (misalnya, Modul **Tampilkan Keranjang**, Modul **Hitung Diskon**, dan Modul **Proses Checkout**) semuanya mengakses dan bergantung pada satu **variabel *state* global** yang menyimpan seluruh isi keranjang belanja pengguna. Jika struktur data keranjang di *state* global diubah, semua modul ini berpotensi rusak.
Content Coupling	Modul `Admin_Produk` dan `Laporan_Penjualan`	Modul `Laporan_Penjualan` secara langsung mengakses dan memodifikasi variabel internal (misalnya, menghapus *cache* atau mengubah *flag* status) di dalam modul `Admin_Produk` alih-alih menggunakan metode publik yang disediakan. Ini adalah yang terburuk karena melanggar enkapsulasi.